Technical Journey: Variant Calling and Benchmarking Pipeline Development

Group 5
Mohammed A. Ahmed, Yasmine M. Elgohary, Ahmad K. Ahmad

Purpose of This Document

This document records technical development, troubleshooting steps, and iterative corrections made during the implementation of the variant calling and benchmarking pipeline for our final project. It complements the scientific report by documenting decisions, corrections, and validation steps that were necessary to ensure a correct and reproducible workflow but are not central to result interpretation.

---

1. Initial Data Inspection and First Pipeline Attempt

1.1 Input data

The project began with the GIAB HG002 whole-exome sequencing BAM file, which was already aligned to an earlier human reference build.

1.2 Initial BAM-to-FASTQ conversion issue

In the initial attempt, the BAM file was directly converted to FASTQ format and used for downstream processing. Because the BAM file was coordinate-sorted, this conversion resulted in the loss of a substantial fraction of paired-end reads. This issue was identified by observing unexpectedly reduced read counts and inconsistent pairing in downstream steps.

---

2. Correction: Name-Sorted BAM and Re-extraction of FASTQs

To address the read-pair loss, the BAM file was first name-sorted prior to FASTQ extraction. Paired-end FASTQs were then regenerated from the name-sorted BAM, preserving read pairing and enabling correct realignment.

This correction prompted a full restart of the downstream pipeline to ensure consistency.

---

3. Realignment to GRCh38 and Deduplication

3.1 Reference preparation

The GRCh38 reference genome was downloaded and indexed using 'samtools', 'Picard', and 'BWA' to ensure compatibility across tools.

3.2 Alignment

The regenerated FASTQs were aligned to GRCh38 using 'bwa mem' with appropriate read-group information. The resulting SAM file was converted to a coordinate-sorted BAM and indexed.

3.3 Deduplication

Duplicate reads were marked using 'Picard MarkDuplicates'. This step was completed prior to any depth estimation or variant calling.

---

4. Chromosome 22 Subsetting

To reduce computational complexity and align with the benchmarking strategy, the deduplicated BAM was subset to chromosome 22 (chr22). All subsequent analyses were performed on this chr22-only BAM.

---

5. Depth Estimation and Discovery of WES-Specific Bias

5.1 Initial genome-wide depth estimation

Depth was initially calculated genome-wide across chr22, yielding an average coverage of approximately 14×. This value was inconsistent with expectations for WES data.

5.2 Correction using exome capture regions

Depth was recalculated using the Agilent SureSelect V5 exome capture BED file downloaded from the Agilent website, which corresponds to the capture design used in the GIAB HG002 whole-exome sequencing dataset. Restricting depth calculations to these assay-specific regions on chromosome 22 revealed a baseline coverage of approximately 215× across about 1.09 million targeted bases, in sharp contrast to the much lower genome-wide estimate.

This demonstrated that genome-wide depth substantially underrepresents effective WES coverage and motivated a revised downsampling strategy.

---

6. Downsampling Strategy and Validation

Using the BED-restricted baseline coverage, subsampled BAM files were generated at approximately 80×, 40×, 10×, and 2× using Picard DownsampleSam with fixed random seeds and

the ConstantMemory strategy. Downsampling probabilities were computed as P = target_coverage / baseline_coverage to preserve relative read representation.

Depth was recalculated for each subsampled BAM using the same exome-restricted approach to confirm that the achieved coverage levels matched the intended targets.

---

7. BQSR and Variant Calling Iterations

Base quality score recalibration was performed using dbSNP138 as known sites. AnalyzeCovariates plots were inspected to confirm successful recalibration.

Variants were then called using 'GATK HaplotypeCaller' in GVCF mode for each coverage level, followed by 'GenotypeGVCFs' to produce annotated VCF files. All calls were restricted to chr22, consistent with the BAM inputs.

---

8. Variant Stratification, Metric Exploration, and Filtering Decisions

8.1 SNP and INDEL separation

Each VCF was split into SNP and INDEL subsets to allow metric-specific assessment and filtering.

8.2 Known versus novel variant stratification

Variants were stratified by novelty using the presence or absence of dbSNP rsIDs. Density plots of quality metrics were generated separately for known and novel variants across coverage levels.

8.3 Filter threshold selection

Metric distributions and summary reports were used to guide filtering decisions across all quality metrics. For each metric, density plots were generated stratified by sequencing coverage, variant type (SNPs and INDELs), and known versus novel variants. This framework was used to review the behavior of standard GATK hard filters and to select coverage-aware thresholds. Filtering decisions were informed by the observed enrichment of novel variants in low-quality tails.

Filtering was applied using GATK VariantFiltration for both SNP and INDEL callsets, with standard hard-filter expressions (for example QD, MQ, MQRankSum, FS, SOR, ReadPosRankSum, InbreedingCoeff) combined with a coverage-specific maximum DP filter. Because depth scales with sequencing coverage, DP thresholds were computed per coverage level using mean + 5 SD and applied separately for SNPs and INDELs. Summary reports were then generated for each filtered VCF, including total, PASS, and filtered variant counts and per-filter-tag contributions stratified by known and novel variants. PASS-only VCFs were carried forward to downstream benchmarking.

---

9. Benchmarking Environment Setup Challenges

9.1 Conda environment incompatibility

Installation of 'hap.py' in the primary ngs1 Conda environment failed due to Python version incompatibility. 'hap.py' required Python 2.7, while ngs1 was pinned to Python 3.10.

9.2 Dedicated benchmarking environment

A separate Conda environment named 'happyenv' was created specifically for benchmarking. 'hap.py', 'RTG Tools', and required dependencies were installed and verified in this environment.

---

10. Benchmarking Workflow Validation

The GIAB HG002 GRCh38 truth VCF and confident regions BED were downloaded and restricted to chr22. Confident regions were intersected with the exome capture BED to restrict benchmarking to confident, assay-relevant regions.

An 'RTG SDF' reference was generated from GRCh38 to enable the 'vcfeval' engine. 'hap.py' was then run for each coverage level and variant type using pass-only variants.

---

11. Metric Extraction and Reporting

Benchmarking outputs were parsed to extract precision, recall, F1 score, genotype concordance, and false positives per megabase. All metrics were compiled into a unified TSV summary and used to generate plots for downstream interpretation.

---

12. Final Pipeline State

At the conclusion of the project, the pipeline consisted of a fully reproducible bash workflow, validated depth estimation and downsampling, coverage-aware filtering, and benchmarking against GIAB truth data following published best practices.